

오늘의 작업기록 — 2026년 7월 10일

애노테이션의 정체 — @는 어떻게 스프링에게 읽히는가

(진도를 나가려다 개념이 발목을 잡았고, 그 발목을 제대로 푼 날)

한눈에

항목	내용
계획	예외→HTTP 번역(핸들러)
실제	애노테이션 3대 질문을 파고들 + README 작성 + 공모전 참가 판단 → 핸들러는 이월
산출	ledger-api/README.md 커밋 · 7/9 파일에 HealthController 풀코드 + 답변 보강
★ 개념	@는 "조회 가능한 명칭" · 종족은 @interface 로 태어날 때 결정 · import는 종족을 못 바꾼다
판단	관광데이터 공모전(7/21 접수, 7월말~9월 개발) 참가하지 않음 — 8월 WAL·SQLD와 정면충돌, 도메인 무관
규칙 추가	작업파일에는 다루는 코드의 풀코드를 항상 병기한다 (과거 파일을 뒤지지 않게)

핸들러를 치러 앉았다가 § 3(어제 예약한 애노테이션 질문)에서 걸렸다. 그냥 넘어갈 수도 있었지만 — 오늘 쓸 문법(@ExceptionHandler)이 정확히 그 애노테이션이라, 모르는 채로 치면 복불이 된다. 그래서 뿌리를 찼다. 진도는 못 나갔지만, 오늘 판 것이 내일 핸들러를 "이해하고 치는" 코드로 만든다.

1. ★ 애노테이션 3대 질문 (오늘의 본체)

기준 코드 — HealthController 전문 (7/6 작성. 질문 셋이 전부 이 파일 애기다)

```
package com.ledger.api;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HealthController {

    @GetMapping("/health")
    public String health() {
        return "ledger-api alive";
    }
}
```

Q1. mini-ledger엔 @가 없었는데, 스프링에선 왜 필요한가?

달라진 건 자바가 아니라 "호출자"다. - mini-ledger: 내가(Main이) bank.transfer(...) 를 직접 호출했다. 누가 뭘 부를지 내가 다 아니까 아무한테도 알릴 필요가 없었다. - 스프링: 스프링이 내 메서드를 대신 호출한다. 그런데 스프링은 내 머릿속을 못 읽는다 — "어느 클래스가 HTTP 담당인지, 어느 URL엔 어느 메서드인지"를 내가 알려줘야 한다. 그 알려주는 언어가 @ 다. mini-ledger에 @가 없던 이유는 문법이 달라서가 아니라, 읽을 주체(스프링)가 없었기 때문이다.

Q2. 스프링이 @를 어떻게 "읽어내는가"? (오늘 새로 판 것)

애노테이션은 주석이 아니다. 자바는 클래스에 붙은 애노테이션을 기계가 조회할 수 있는 명칭로 저장한다 — 컴파일된 .class 파일 안에 그 명칭이 같이 박힌다. 그래서 실행 중에 프로그램이 "이 클래스에 무슨 명칭이 붙어 있냐"고 물어볼 수 있다.

부팅 시퀀스로 보면:

```

서버 부팅
└─ 스프링 스캐너가 프로젝트의 클래스들을 하나씩 돌면서 묻는다
    "너 @RestController 명칭 불러 있냐?"
    └─ 아니오 → 통과 (관심 없음)
        └─ 예 → ① 이 클래스의 객체를 만들어 등록
            ② 이 클래스의 메서드들을 훑으며 다시 묻는다
                "너 @GetMapping 명칭 불러 있냐? 값이 뭐냐?"
                └─ 예, 값은 "/health"
                    → URL 표에 한 줄 기록: GET /health → health()

요청 도착 (GET /health)
└─ URL 표에서 찾아 → health() 호출 → return 값을 응답 본문으로

```

→ 부팅 = 전 클래스 명칭 검사. @는 장식이 아니라 **조회 가능한 데이터**다. 내가 등록 코드를 한 줄도 안 썼는데 컨트롤러가 등록되는 이유의 전부가 이것이다.

Q3. Map은 클래스인데 RestController는 왜 애노테이션인가? 둘 다 import인데?

중족은 태어날 때(정의될 때) 정해진다. 각자의 출생신고(정의 소스)를 보면 다르다:

```

public interface Map<K,V> { ... } // Map의 정의: 재료(인터페이스/클래스 계열)
public @interface RestController { ... } // RestController의 정의: @interface = 애노테이션

```

@interface 라는 **별도 키워드**로 정의된 것만 애노테이션이다. 그리고 **import** 는 "이 이름을 이 파일에서 쓰겠다"고 **주소록에 올리는 행위**일 뿐 — 이름이 가리키는 실체의 종족은 이미 정해져 있고, import가 그걸 바꾸지 못한다.

그래서 **사용법은 중족을 따른다**:

	Map (재료)	RestController (스티커)
정의 키워드	interface / class	@interface
쓰는 법	Map<String,Long> m = new HashMap<>(); — 만들어서 쓴다	@RestController — 클래스 위에 붙인다
반대로 쓰면	@Map → 컴파일 에러	new RestController() → 컴파일 에러

2. README 작성 (ledger-api)

repo를 열었을 때 설명이 없으면, 채용담당자가 코드를 읽기 전에 이탈한다. 첫인상 파일을 채웠다.

```

cd ~/ledger-api && cat > README.md << 'EOF'
# ledger-api
결제·정산 백엔드를 목표로 한 학습 프로젝트. 순수 Java로 만든 결제 원장(mini-ledger)을
Spring Boot HTTP API로 확장 중.

## 현재 기능
- `GET /health` — 서버 생존 확인
- `POST /transfer` — 계좌 이체 (수수료 정산 포함)
  - **幂등성**: 같은 idempotencyKey 재요청 시 중복 처리 방지 (Stripe/Toss 패턴, HTTP 검증 완료)
  - **복식부기 검증**: 매 이체마다 시스템 전체 잔액 합 = 0 확인

## 스택
Java 21 · Spring Boot · Gradle
EOF
git add README.md && git commit -m "add README" && git push
# 판정: 7c7b7da  main -> main

```

작성 원칙: 기능 나열이 아니라 **보증(guarantee)**을 앞세운다. "이체 됨"이 아니라 "중복 요청이 와도 한 번만 처리됨(幂등성)", "매 이체마다 돈이 새지 않음을 검증함(복식부기)" — 결제 백엔드에서 채용담당자가 보는 건 기능 목록이 아니라 이 보증들이다.

3. 판단 — 관광데이터 공모전 참가하지 않기로

한국관광공사 공모전(웹·앱 구현 부문, 접수 7/21, 개발 7월말~9월, 상금 최대 1000만원) 참가를 검토했고, **참가하지 않기로 결정했다**.

기준	판단
도메인 적합성	관광 OpenAPI 매시업 = 프론트+데이터. 내 목표는 결제·정산 백엔드 . 면접에서 묻는 것(동시성·정합성·장애 시 돈 보호)과 겹치는 면적이 작다
일정 충돌	개발 기간 7월말~9월 = 8월 Layer 0(WAL, 최종량) + SQLD(8/22) 구간과 정면충돌. 포트폴리오의 심장이 밀린다
현역 제약	팀 협업 일정을 부대 스케줄에 맞출 수 없고, 1인이면 두 달 풀타임급 부담을 폰+VPS로 진다
대안 우위	같은 두 달을 ledger-api에 WAL을 엮는 데 쓰면 "장애가 나도 돈이 안 새는 결제 서버"가 나온다 — 핀테크 채용엔 이쪽이 압도적으로 강하다

메타 교훈: 새 프로젝트는 초기에 항상 재밌고(진도 나가는 느낌), 그래서 **정체감이 들 때 특히 유혹적**이다. 하지만 "복기만 하는 것 같다"는 불안의 해법은 새 트랙을 여는 게 아니라 **원래 트랙의 다음 커밋**이다. 분산은 불안의 해결책이 아니라 증상이다.

4. 규칙 추가 — 작업파일에 풀코드 병기

오늘 애노테이션 질문을 하면서 `@GetMapping("/health")` 가 든 파일 전문을 보려고 **과거 작업기록을 뒤져야 했다**. 이걸 기록 방식의 결함이다.

→ 앞으로 작업파일에서 다루는(질문·복기·교정 대상) 코드는, 첫 등장이 아니어도 그 파일에 풀코드로 병기한다. 파일 하나로 그날 학습이 완결되게. (7/9 파일에 HealthController 전문과 위 Q2·Q3 답변을 소급 삽입 완료.)

5. 다음

- **핸들러(예외→HTTP 번역) 이월** — 오늘 판 애노테이션 지식이 그대로 `@ExceptionHandler`에 쓰인다.
- 재진입 명령: 부팅 → `curl -i` 로 500 관찰 → 핸들러 → 4xx 확인 → 200 회귀 → 커밋.

6. 검증 질문 (답 적지 말 것)

1. mini-ledger엔 @가 없었다. 스프링에선 왜 필요한가? (호출자 관점)
2. 스프링은 부팅 시 @를 "어떻게" 읽어내나? URL 표가 만들어지는 과정을 순서대로.
3. `Map` 과 `RestController` 의 종족이 다른 이유는? 정의 소스의 어떤 키워드가 그것을 결정하나?
4. `new RestController()` 와 `@Map` 은 각각 왜 애러인가?
5. README에서 기능이 아니라 "보증"을 앞세운 이유는?
6. 공모전을 거절한 근거 넷은? "정체감이 들 때 새 트랙이 유혹적"인 이유는?

7. 회고

계획대로면 오늘은 핸들러의 날이었다. 그런데 어제 예약해둔 § 3 질문에서 걸렸고, 걸린 김에 끝까지 봤다. 결과적으로 이게 옳았다고 본다 — 오늘 쓸 문법이 하필 `@ExceptionHandler` , 즉 애노테이션이었기 때문이다. "@가 뭔지 모르는 채로 `@ExceptionHandler`를 치는 것"은 복붙이지 학습이 아니다.

특히 두 번째 질문("스프링이 @를 어떻게 읽어내나")은 내가 스스로 밀고 들어간 질문이었다. 어제는 "@는 스프링이 읽는 스티커"에서 멈췄는데, 오늘은 "읽는다는 게 물리적으로 무슨 뜻이냐"까지 갔다. 답 — 컴파일된 클래스에 조회 가능한 명찰로 박히고, 부팅이 그 명찰들을 전수 검사하는 것 — 을 알고 나니 마법 ②③이 마법이 아니라 절차로 보인다. 세 번째 질문(Map vs RestController의 종족)도 마찬가지다. `@interface` 라는 키워드 하나를 보고 나니 "왜 애는 붙이고 재는 만드나"가 규칙이 아니라 당연한 귀결이 됐다.

그리고 오늘 두 가지를 거절했다. 하나는 공모전 — 재밌어 보이고 상금도 크지만, 8월 WAL을 밀어내는 대가가 너무 컸다. "며칠째 복기만 하는 것 같다"는 불안이 정확히 이런 제안을 매력적으로 만든다는 것도 알아차렸다. 분산은 불안의 해결책이 아니다. 다른 하나는 나 자신의 기록 방식 — 과거 파일을 뒤져야 코드를 볼 수 있다면 그 파일은 미완이다. 규칙을 고쳤다.

핸들러는 이월됐지만, 오늘 판 것 위에서 내일은 "이해하고 치는" 코드가 된다.

한 줄: @는 조회 가능한 명찰이고 부팅은 전수 명찰 검사다 — 진도 대신 그 뿌리를 찼고, 새 트랙(공모전)은 거절했다.